READ FIRST

One master. Many workers.

Combine the questions once, distribute the frozen work, then reunite episodes and activations before creating labels.

Stage	Runs on	Produces
Prepare	Coordinator, once	Master manifest + frozen work chunks
Collect	GPU workers	Episodes + activation checkpoints
Merge	Coordinator	One ordered set of collected results
Split / export	Coordinator	Aligned partitioned episodes + tensors
Claude label	Labeling machine	Train / validation behavior labels
Archive	Coordinator	Complete S3 collection bundle

Master is made before distribution. Prepare combines MuSiQue and synthetic question payloads into manifest.json. Post-collection merge combines worker results; it does not create a new question pool.

What this guide completes

A mixed collection, aligned activation exports, and Claude-based development labels. SAE training, probe training, feature selection, and causal interventions are later workflows.

Current boundary

The Claude labeling CLI permits **train and validation only**. Test and intervention episodes are collected/exported, but their Claude labeling is blocked by the current CLI. Do not describe this guide as completing labels for all partitions.

Expected question pool

19,938 MuSiQue train + 2,417 MuSiQue validation + a target of 80,004 synthetic = **102,359 questions**. This total assumes generation meets its target. Scientific exclusions can reduce collected episodes; adopt/retain eligibility reduces usable binary labels further.

00 / ALL MACHINES

Use the same code and shell setup

Run commands from the repository root in the capstone environment. Start a tmux session before long GPU or labeling work.

Code prerequisite: use one integrated checkout containing the distributed and Claude labeling implementations from main, plus this branch's synthetic code and mixed config. The inspected synthetic branch lacks scripts/distributed_collection.py. Resolve integration before preparing the production manifest.

```
tmux new -s mixed-collection
# Inside tmux, from the repository root:
conda activate capstone
export PYTHONPATH=src
export AWS_PROFILE=school
export COLLECTION_RUN=mixed_opus5_full_001
export RUN_DIR="$PWD/artifacts/collection/$COLLECTION_RUN"
export COLLECTION_CONFIG="$PWD/configs/collection/mixed_opus5_full_001.yaml"
export S3_RUN="s3://dats-capstone/shared/group3/artifacts/collection/$COLLECTION_RUN"
export GEN_DIR="$PWD/data/MuSiQue/synthetic/runs/opus5-v3-full-001/generation"

aws sso login --profile school --use-device-code
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_SECURITY_TOKEN
export aws_access_key_id=""
export aws_secret_access_key=""
export aws_session_token=""
aws sts get-caller-identity --profile school
```

Repeat these exports on every machine or new shell. Empty lowercase credential variables prevent load_dotenv() from supplying old explicit credentials; boto3 then uses the normal profile chain. Successful STS identity does not by itself establish Bedrock model permission.

```
test -f scripts/distributed_collection.py
test -f scripts/run_claude_validation.py
test -f "$COLLECTION_CONFIG"
git rev-parse HEAD
```

Use the same integrated commit, config, and downloaded manifest on all workers. Collection needs access to the pinned Gemma/Qwen checkpoints, the appropriate environment, and a compatible GPU. Merge/export need sufficient RAM and disk for checkpoints plus merged and exported tensors.

CONFIG / PART 1

Pinned local collection models

The complete config is configs/collection/mixed_opus5_full_001.yaml. These are its role settings; retain them consistently across workers.

```
roles:
  solver:
    id: google/gemma-3-4b-it
    revision: 093f9f388b31de276ce2de164bdc2081324b9767
    loader: gemma3
    dtype: bfloat16
    device: cuda:0
    generation:
      max_new_tokens: 64
      do_sample: false
  critic:
    id: Qwen/Qwen3-4B-Instruct-2507
    revision: cdbee75f17c01a7cc42f958dc650907174af0554
    loader: qwen3
    dtype: bfloat16
    device: cuda:0
    generation:
      max_new_tokens: 1024
      do_sample: false
  validator:
    id: google/gemma-3-4b-it
    revision: 093f9f388b31de276ce2de164bdc2081324b9767
    loader: gemma3
    dtype: bfloat16
    device: cuda:0
    generation:
      max_new_tokens: 4
      do_sample: false
```

Solver and validator use Gemma 3 4B; the critic uses Qwen3 4B. These are local collection models. Claude is used later for behavior labels; it is not the model running these episodes.

Generation's API concurrency setting does not control local episode collection. Scale collection by assigning different work chunks to GPU machines.

CONFIG / PART 2

Mixed inputs and frozen settings

This config includes both original MuSiQue splits and the synthetic release. It does not replace MuSiQue with synthetic-only input.

```
dataset:
  inputs:
  - source: musique
    version: musique-clean-local-v1
    source_split: train
    path: data/MuSiQue/clean/train.jsonl
  - source: musique
    version: musique-clean-local-v1
    source_split: validation
    path: data/MuSiQue/clean/validation.jsonl
  - source: synthetic
    version: opus5-v3-full-001
    path: data/MuSiQue/synthetic/runs/opus5-v3-full-001/generation/accepted.jsonl
    generation_provenance:
      run_id: opus5-v3-full-001
      artifacts_dir: data/MuSiQue/synthetic/runs/opus5-v3-full-001/generation
      record_join_key: id -> provenance.jsonl record_id
  intervention:
    fraction: 0.1
    seed: 42
  split:
    proportions:
      train: 0.8
      validation: 0.1
      test: 0.1
    seed: 43
collection:
  protocol_version: v2
  active_conditions:
  - natural
  a2_presentations:
  - candidates
  layers:
  - 4
  - 8
  - 17
  - 25
  - 29
  - 30
  - 31
  - 32
  - 33
  seed: 42
output:
  run_name: mixed_opus5_full_001
  chunk_size: 50
```

MuSiQue version is a local snapshot label, not an asserted Hub revision. Prepare hashes the actual input files. Synthetic generation provenance is a release reference: the importer does not automatically join its sidecar into each record.

The example reserves 10% intervention, then allocates the remaining pool 80/10/10 to train/validation/test: approximately 72/9/9/10 overall. Assignment respects explicit families and source/hop/candidate-order strata. Original MuSiQue validation is an origin tag, not a permanently fixed final validation partition in this workflow.

01 / COORDINATOR

Finish generation, then stage the inputs

Do not prepare while accepted.jsonl is still being written. The manifest freezes the exact payloads available at preparation time.

If synthetic generation has not yet completed

```
# Paid calls. Run once on the generation machine, in tmux.
python -u scripts/generate_synthetic_musique.py \
  --config configs/synthetic/opus5_v3_full.yaml

# Upload only after the run has stopped writing.
aws s3 sync \
  data/MuSiQue/synthetic/runs/opus5-v3-full-001/ \
  s3://dats-capstone/shared/group3/data/MuSiQue/synthetic/runs/opus5-v3-full-001/ \
  --profile school
```

This branch's full-run config targets 80,004 structurally accepted questions with 256 generation workers. Its automatic Claude dataset-redundancy audit is disabled. Keep validation, exact deduplication, and generation provenance artifacts.

On the coordinator

```
aws s3 sync \
  s3://dats-capstone/shared/group3/data/MuSiQue/synthetic/runs/opus5-v3-full-001/ \
  data/MuSiQue/synthetic/runs/opus5-v3-full-001/ \
  --profile school

# Only if either original input is missing:
if [ ! -f data/MuSiQue/clean/train.json ] || \
   [ ! -f data/MuSiQue/clean/validation.json ]; then
  python scripts/get_musique_dataset.py
fi
cp data/MuSiQue/clean/train.json data/MuSiQue/clean/train.jsonl
cp data/MuSiQue/clean/validation.json data/MuSiQue/clean/validation.jsonl

test -f "$GEN_DIR/accepted.jsonl"
cat "$GEN_DIR/summary.json"
```

The existing clean .json files are JSONL. Copies with .jsonl extensions make the master importer select the correct parser. Confirm generation's status and actual accepted count; a request cap can leave the target unmet.

02 / COORDINATOR

Create the master exactly once

The coordinator needs all three input files. GPU workers only need the resulting manifest and config.

```
mkdir -p "$RUN_DIR"
cp "$COLLECTION_CONFIG" "$RUN_DIR/collection.yaml"

python scripts/distributed_collection.py prepare \
  --config "$RUN_DIR/collection.yaml" \
  --num-chunks 8 \
  --output "$RUN_DIR/manifest.json"

aws s3 cp "$RUN_DIR/manifest.json" "$S3_RUN/manifest.json" \
  --profile school
aws s3 cp "$RUN_DIR/collection.yaml" "$S3_RUN/collection.yaml" \
  --profile school
```

What prepare freezes

- Namespaced IDs such as synthetic:2hop__synthetic_... and musique:2hop__....
- Exact question payloads, input file hashes, and source/version metadata.
- Intervention reservation, per-question collection seeds, candidate order, and eight work slices.
- The split proportions and seed. Ordinary train/validation/test membership is assigned after merge, without using behavioral outcomes.

Eight chunks are eight work assignments, not eight concurrent API requests. output.chunk_size: 50 means a checkpoint batch of 50 questions within each assignment.

Check the frozen question counts

```
python - <<'CHECK'
import json, os
from collections import Counter
from pathlib import Path
m = json.loads((Path(os.environ["RUN_DIR"])/"manifest.json").read_text())
print("Master questions:", len(m["questions"]))
print("By source:", dict(Counter(q["dataset_source"] for q in m["questions"])))
print("Chunks:", len(m["chunks"]))
print("Manifest hash:", m["manifest_sha256"])
CHECK
```

Do not edit the manifest or prepare a different one per worker. Change sources, settings, or membership only by preparing a new experiment/run.

03 / GPU WORKERS

Download one master. Own different chunks.

Example: worker A owns chunks 0–3; worker B owns chunks 4–7. Never run two owners of the same chunk simultaneously.

On both workers: run page 2 setup, then download

```
mkdir -p "$RUN_DIR"
aws s3 cp "$S3_RUN/manifest.json" "$RUN_DIR/manifest.json" \
  --profile school
aws s3 cp "$S3_RUN/collection.yaml" "$RUN_DIR/collection.yaml" \
  --profile school
```

Choose the assignment on each worker

```
# Worker A only:
OWNED_CHUNKS=(0 1 2 3)

# Worker B instead uses this assignment:
# OWNED_CHUNKS=(4 5 6 7)
```

Run the same collection command on each worker

```
python -u scripts/collect_activations.py \
  --config "$RUN_DIR/collection.yaml" \
  --manifest "$RUN_DIR/manifest.json" \
  --chunk "${OWNED_CHUNKS[@]}"
```

Models load once per invocation. The worker runs Solver A1, the configured natural Critic interaction, Solver A2, and activation collection. Outputs are saved beneath chunks/000000/, chunks/000001/, and so on.

If that worker is interrupted

```
python -u scripts/collect_activations.py \
  --config "$RUN_DIR/collection.yaml" \
  --manifest "$RUN_DIR/manifest.json" \
  --chunk "${OWNED_CHUNKS[@]}" \
  --resume
```

Resume reuses committed checkpoints and skips matching completed work. An unfinished checkpoint batch may repeat. Keep local storage; S3 upload does not happen automatically. For one machine, use OWNED_CHUNKS=(0 1 2 3 4 5 6 7).

04 / WORKERS → S3 → COORDINATOR

Upload completed chunks, marker last

A completion marker is the signal that a chunk is ready to merge. Upload data first, then complete.json.

On each worker after its assignment completes

```
# Recreate OWNED_CHUNKS if you opened a new shell.
for chunk in "${OWNED_CHUNKS[@]}"; do
  chunk_name=$(printf '%06d' "$chunk")
  chunk_dir="$RUN_DIR/chunks/$chunk_name"
  test -f "$chunk_dir/complete.json" || exit 1

  aws s3 sync "$chunk_dir/" "$S3_RUN/chunks/$chunk_name/" \
    --exclude complete.json --exclude '*.tmp/*' \
    --profile school || exit 1

  aws s3 cp "$chunk_dir/complete.json" \
    "$S3_RUN/chunks/$chunk_name/complete.json" \
    --profile school || exit 1
done
```

Upload only your assigned chunks. If a transfer fails, repeat it after the worker has stopped writing. Stop the old owner before reassigning a chunk. This workflow does not reconcile competing attempts.

On the coordinator, after every assigned chunk is uploaded

```
aws s3 sync "$S3_RUN/chunks/" "$RUN_DIR/chunks/" \
  --profile school

# This eight-chunk example expects all eight completion markers.
for chunk in 0 1 2 3 4 5 6 7; do
  chunk_name=$(printf '%06d' "$chunk")
  test -f "$RUN_DIR/chunks/$chunk_name/complete.json" || exit 1
done
```

Marker presence is a quick readiness check, not validation. The merge command checks completion hashes, master identity, membership, seeds, candidate order, coverage, and tensor/episode alignment.

05 / COORDINATOR

Merge results, then export partitions

This is the second kind of merge: it reunites collected worker results, not source question files.

```
python scripts/distributed_collection.py merge \
  --manifest "$RUN_DIR/manifest.json"

python scripts/distributed_collection.py split \
  --manifest "$RUN_DIR/manifest.json" \
  --config "$RUN_DIR/collection.yaml"
```

Expected layout

```
$RUN_DIR/
  merged/
    collection.json
    activations/raw/layer_NN/all*.pt
    results/raw/all/interactions.jsonl
  prepared/
    partitions.csv
    split.json
    activations/mixed_opus5_full_001/layer_NN/
    activations/mixed_opus5_full_001_intervention/layer_NN/
    results/mixed_opus5_full_001/
      train/interactions.jsonl
      validation/interactions.jsonl
      test/interactions.jsonl
      split_manifest.csv
      collection_status.json
    results/mixed_opus5_full_001_intervention/
      intervention/interactions.jsonl
      contexts.jsonl
      collection_status.json
```

- Merge orders records by the master and rebuilds activation indices.
- Split assigns non-intervention questions without looking at labels or outcomes, then exports aligned tensors and interactions.
- Label the final prepared/results files, not the internal merged/raw files. Their indices refer to the final partition tensors.
- Inspect merged/collection.json and both final collection_status.json files. Scientific exclusions are recorded; missing work chunks are not scientific exclusions.

Fresh merge and prepared destinations are required. Failed builds use staging directories; do not delete or overwrite successful outputs to force a rerun. Preserve the original manifest, chunks, and frozen partition mapping.

06 / LABELING MACHINE

Create Claude behavior labels

Claude judges whether A2 follows the Critic or retains A1. It does not judge whether the synthetic question or its answer is factually correct.

Use page 2's refreshed school profile setup. The labels below explicitly request the account-tested Opus ID; the script otherwise defaults to Sonnet. These are separate from both the disabled redundancy audit and prepare_behavior_v01.py's deterministic labels.

Preview ten train episodes: no API calls

```
python scripts/run_claude_validation.py \
  --episodes "$RUN_DIR/prepared/results/$COLLECTION_RUN/train/interactions.jsonl" \
  --partition train --model-id us.anthropic.claude-opus-5 \
  --effort medium --max-tokens 8000 \
  --output "$RUN_DIR/claude_labels/train" \
  --limit 10 --dry-run
```

Paid labeling: train and validation

```
for partition in train validation; do
  python -u scripts/run_claude_validation.py \
    --episodes "$RUN_DIR/prepared/results/$COLLECTION_RUN/$partition/interactions.jsonl" \
    --partition "$partition" \
    --model-id us.anthropic.claude-opus-5 \
    --effort medium --max-tokens 8000 \
    --output "$RUN_DIR/claude_labels/$partition" || exit 1
done
```

Do not add --limit to the full run. The validation command must explicitly select --partition validation; the default is train. Output directories must be new. Dry-run does not create them or validate live Bedrock access.

Result	Binary export
Adoption	primary_target = 1; eligible_primary = true
Retention	primary_target = 0; eligible_primary = true
Agreement / out	Excluded from binary training labels, with reasons
API / parse / truncation failure	Technical exclusion, not a behavioral decision

Current labeling is serial within each invocation and has no --concurrency flag. Generation/audit concurrency does not speed it up. Noncommittal-critic exclusions can be decided by rule and are identified by label_source. Opus's output budget also covers reasoning; a pilot is needed to measure actual speed and successful response rate.

07 / LABELING RECOVERY

Inspect errors before calling the run complete

A completed process may still contain technical exclusions. Preserve original outputs and retry only failed work when the script supports it.

Inspect both summaries

```
cat "$RUN_DIR/claude_labels/train/summary.json"
cat "$RUN_DIR/claude_labels/validation/summary.json"
```

Check n_episodes, label_counts, n_technical_errors, error_reasons, label_sources, and token usage. Agreement and out are legitimate exclusions. Technical failures should not silently count as retention. No fixed adopt/retain yield is guaranteed.

Retry technical failures from a finished train run

```
python -u scripts/run_claude_validation.py \
  --episodes "$RUN_DIR/prepared/results/$COLLECTION_RUN/train/interactions.jsonl" \
  --partition train --model-id us.anthropic.claude-opus-5 \
  --effort medium --max-tokens 8000 \
  --retry-technical-errors-from "$RUN_DIR/claude_labels/train" \
  --output "$RUN_DIR/claude_labels/train_retry_001"

python scripts/summarize_claude_validation.py \
  --runs "$RUN_DIR/claude_labels/train" \
         "$RUN_DIR/claude_labels/train_retry_001" \
  --output "$RUN_DIR/claude_labels/train_combined"
```

Do this only when technical failures exist. For validation, substitute validation in the episode path, --partition, and output directories. Combine runs for the same cohort/source; do not use this retry combiner to mix train and validation.

No automatic labeling resume: a fatal stop may leave raw_results.jsonl without a final summary. Preserve it, refresh credentials, and assess recovery before resubmitting the whole cohort; repeating into a new directory can bill already completed episodes again. The finished-run technical retry command is not a general crash-resume command.

Collection resume is different: collect_activations.py --resume is supported for matching checkpoints. Neither synthetic generation nor this Claude labeling CLI has the same automatic resume behavior.

08 / ARCHIVE & HANDOFF

Save the bundle. Keep the boundaries clear.

Upload after writers have stopped. Keep generation artifacts as well as collection artifacts so the questions, episodes, tensors, and decisions remain traceable.

```
# On the coordinator after final processing:
aws s3 sync "$RUN_DIR/" "$S3_RUN/" --profile school

# On a local computer, from its repository root:
aws sso login --profile school --use-device-code
aws s3 sync \
  s3://dats-capstone/shared/group3/artifacts/collection/mixed_opus5_full_001/ \
  artifacts/collection/mixed_opus5_full_001/ \
  --profile school
```

What downstream users receive

- Frozen question and split identity: manifest.json, collection.yaml, prepared/partitions.csv, and split.json.
- Aligned prepared/results and prepared/activations for ordinary and intervention runs.
- Claude train/validation binary_labels.csv, raw_results.jsonl, exclusions.jsonl, excluded_episodes.json, selected_episodes.json, and summary.json.
- Use combined outputs for a cohort when a technical-error retry was combined. Targets are primary_target / eligible_primary, not deterministic strict_target / eligible_strict.
- Downstream alignment must join IDs against the matching frozen manifest and activation indices, with excluded_episodes.json accounted for. Do not substitute new labels into an old results/tensor package.

Remaining scope / evidence

The current CLI refuses test and intervention labeling. An explicit later implementation/policy decision is needed before claiming all-partition Claude labels or final held-out evaluation. The failed dataset-redundancy audit remains disabled; behavior labeling does not close semantic question-quality gaps.

This runbook is a command and source review, not a production execution certificate. No GPU episode collection, paid labeling, or live S3 transfer was executed to produce it. Offline checks: 169 synthetic tests passed earlier; this build passed 147 distributed/Claude tests with network blocked. The mixed config also prepared six real sample records using cached main.

Source snapshots: synthetic branch `646da88fe4b7`; cached origin/main `e1ee614ddd6c`. Live main could not be refreshed through local Git; confirm current implementations before deployment. Config: `configs/collection/mixed_opus5_full_001.yaml`.

Verified source references: main's docs/distributed_collection.md, distributed_collection/{dataset,core,cli}.py, collect_activations.py, run_claude_validation.py, summarize_claude_validation.py, and evaluation/{claude_validator,claude_adaptive_client}.py. The config and synthetic settings come from this branch.